

```
IndentationError: expected an indented block after 'for' statement on line 34
```

Explain error

7

```

    ],
    'NAME_HOUSING_TYPE': [
        'House',
        'Apartment',
        'House',
        'House',
        'Apartment',
        'House',
        'Apartment',
        'House'
    ],
    'OCCUPATION_TYPE': [
        'Labor',
        'Manager',
        None,
        'Clerk',
        None,
        'Sales',
        None,
        None
    ]
}

credit_app = pd.DataFrame(data)

# =====
# Missing Values
# =====

print("Missing Values Count")
print(credit_app.isnull().sum())

print("\nMissing Value Percentage")
print(credit_app.isnull().mean() * 100)

# Remove Occupation Column

credit_app = credit_app.drop("OCCUPATION_TYPE", axis=1)

print("\nAfter Removing OCCUPATION_TYPE")
print(credit_app.isnull().sum())

# =====
# Label Encoding
# =====

le = LabelEncoder()

columns_to_encode = [
    'CODE_GENDER',
    'FLAG_OWN_CAR',
    'FLAG_OWN_REALTY',
    'NAME_INCOME_TYPE',
    'NAME_EDUCATION_TYPE',
    'NAME_FAMILY_STATUS',
    'NAME_HOUSING_TYPE'
]

for col in columns_to_encode:
    credit_app[col] = le.fit_transform(credit_app[col])

# =====

```

```

# Prepare Features
# =====

credit_app = credit_app.drop("ID", axis=1)

X = credit_app.drop("FLAG_OWN_CAR", axis=1)

y = credit_app["FLAG_OWN_CAR"]

# =====
# Train Test Split
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# =====
# Train Model
# =====

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

# =====
# Prediction
# =====

predictions = model.predict(X_test)

print("\nPredictions")
print(predictions)

print("\nAccuracy")
print(accuracy_score(y_test, predictions))

# =====
# Predict New Data
# =====

new_data = pd.DataFrame(
    [[
        1,      # CODE_GENDER
        1,      # FLAG_OWN_REALTY
        1,      # CNT_CHILDREN
        55000,   # AMT_INCOME_TOTAL
        0,      # NAME_INCOME_TYPE
        0,      # NAME_EDUCATION_TYPE
        1,      # NAME_FAMILY_STATUS
        0       # NAME_HOUSING_TYPE
    ]],
    columns=X.columns
)

prediction = model.predict(new_data)

print("\nPrediction for New Customer")

```

```
print(prediction)

if prediction[0] == 1:
    print("Customer is predicted to OWN a car.")
else:
    print("Customer is predicted to NOT OWN a car.")
```

Missing Values Count

ID	0
CODE_GENDER	0
FLAG_OWN_CAR	0
FLAG_OWN_REALTY	0
CNT_CHILDREN	0
AMT_INCOME_TOTAL	0
NAME_INCOME_TYPE	0
NAME_EDUCATION_TYPE	0
NAME_FAMILY_STATUS	0
NAME_HOUSING_TYPE	0
OCCUPATION_TYPE	4

dtype: int64

Missing Value Percentage

ID	0.0
CODE_GENDER	0.0
FLAG_OWN_CAR	0.0
FLAG_OWN_REALTY	0.0
CNT_CHILDREN	0.0
AMT_INCOME_TOTAL	0.0
NAME_INCOME_TYPE	0.0
NAME_EDUCATION_TYPE	0.0
NAME_FAMILY_STATUS	0.0
NAME_HOUSING_TYPE	0.0
OCCUPATION_TYPE	50.0

dtype: float64

After Removing OCCUPATION_TYPE

ID	0
CODE_GENDER	0
FLAG_OWN_CAR	0
FLAG_OWN_REALTY	0
CNT_CHILDREN	0
AMT_INCOME_TOTAL	0
NAME_INCOME_TYPE	0
NAME_EDUCATION_TYPE	0
NAME_FAMILY_STATUS	0
NAME_HOUSING_TYPE	0

dtype: int64

Predictions

[1 1]

Accuracy

0.0

Prediction for New Customer

[1]

Customer is predicted to OWN a car.

